Lecture 38

Hamcycle $\leq_p$ *TSP*, Approximation Algorithms, *OptVertexCover*

$Hamcycle \leq_p TSP$

Hamcycle $\leq_p$ *TSP*

- *Hamcycle* $= \{\langle G \rangle \mid G$ is an undirected graph with a hamiltonian cycle$\}$

Hamcycle $\leq_p$ *TSP*

- *Hamcycle* $= \{\langle G \rangle \mid G$ is an undirected graph with a hamiltonian cycle$\}$
- *TSP* $= \{\langle G', c, k \rangle \mid G'$ is complete undirected graph,

Hamcycle $\leq_p$ *TSP*

- *Hamcycle* $= \{\langle G \rangle \mid G$ is an undirected graph with a hamiltonian cycle$\}$
- *TSP* $= \{\langle G', c, k \rangle \mid G'$ is complete undirected graph, c: $V \times V \to \mathbb{N}$ is a cost function,

Hamcycle $\leq_p$ *TSP*

- *Hamcycle* $= \{\langle G \rangle \mid G$ is an undirected graph with a hamiltonian cycle$\}$
- *TSP* $= \{\langle G', c, k \rangle \mid G'$ is complete undirected graph, c: $V \times V \to \mathbb{N}$ is a cost function,
  $k \in \mathbb{N}$,

Hamcycle $\leq_p$ *TSP*

- *Hamcycle* $= \{\langle G \rangle \mid G$ is an undirected graph with a hamiltonian cycle$\}$
- *TSP* $= \{\langle G', c, k \rangle \mid G'$ is complete undirected graph, c: $V \times V \to \mathbb{N}$ is a cost function, $k \in \mathbb{N}$, and G' has a traveling-salesperson tour with cost at most $k\}$

Hamcycle $\leq_p$ *TSP*

- *Hamcycle* $= \{\langle G \rangle \mid G$ is an undirected graph with a hamiltonian cycle$\}$
- *TSP* $= \{\langle G', c, k \rangle \mid G'$ is complete undirected graph, $c\colon V \times V \to \mathbb{N}$ is a cost function, $k \in \mathbb{N}$, and G' has a traveling-salesperson tour with cost at most $k\}$

Hamiltonian cycle

Hamcycle $\leq_p$ *TSP*

- *Hamcycle* $= \{\langle G \rangle \mid G$ is an undirected graph with a hamiltonian cycle$\}$
- *TSP* $= \{\langle G', c, k \rangle \mid G'$ is complete undirected graph, c: $V \times V \to \mathbb{N}$ is a cost function, $k \in \mathbb{N}$, and G' has a traveling-salesperson tour with cost at most $k\}$

Hamcycle $\leq_p$ *TSP*

- *Hamcycle* $= \{\langle G \rangle \mid G$ is an undirected graph with a hamiltonian cycle$\}$
- *TSP* $= \{\langle G', c, k \rangle \mid G'$ is complete undirected graph, c: $V \times V \to \mathbb{N}$ is a cost function, $k \in \mathbb{N}$, and G' has a traveling-salesperson tour with cost at most $k\}$

$\langle G \rangle \to \langle G', c, k \rangle$:

Hamcycle $\leq_p$ *TSP*

- *Hamcycle* $= \{\langle G \rangle \mid G$ is an undirected graph with a hamiltonian cycle$\}$
- *TSP* $= \{\langle G', c, k \rangle \mid G'$ is complete undirected graph, c: $V \times V \to \mathbb{N}$ is a cost function, $k \in \mathbb{N}$, and G' has a traveling-salesperson tour with cost at most $k\}$

$\langle G \rangle \to \langle G', c, k \rangle$:

- G' is a complete graph on $V(G)$.

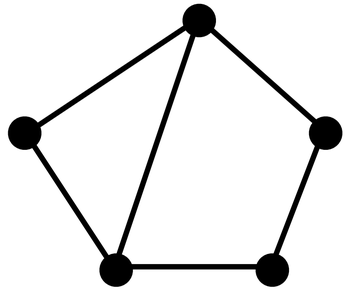
Hamcycle $\leq_p$ *TSP*

- *Hamcycle* $= \{\langle G \rangle \mid G$ is an undirected graph with a hamiltonian cycle$\}$
- *TSP* $= \{\langle G', c, k \rangle \mid G'$ is complete undirected graph, c: $V \times V \to \mathbb{N}$ is a cost function, $k \in \mathbb{N}$, and G' has a traveling-salesperson tour with cost at most $k\}$

$\langle G \rangle \to \langle G', c, k \rangle$:

- G' is a complete graph on $V(G)$.

G

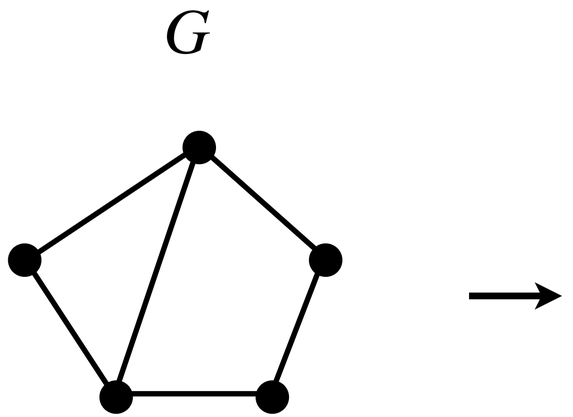


Hamcycle $\leq_p$ *TSP*

- *Hamcycle* $= \{\langle G \rangle \mid G$ is an undirected graph with a hamiltonian cycle$\}$
- *TSP* $= \{\langle G', c, k \rangle \mid G'$ is complete undirected graph, c: $V \times V \to \mathbb{N}$ is a cost function, $k \in \mathbb{N}$, and G' has a traveling-salesperson tour with cost at most $k\}$

$\langle G \rangle \to \langle G', c, k \rangle$:

- G' is a complete graph on $V(G)$.

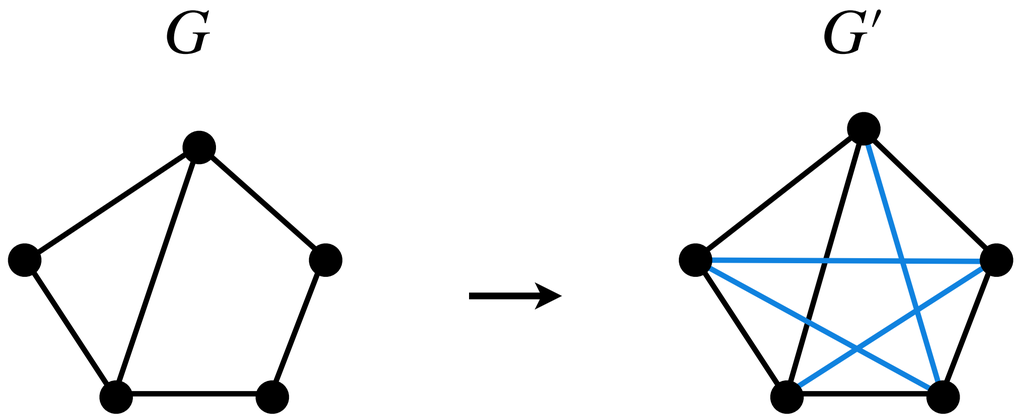


Hamcycle $\leq_p$ *TSP*

- *Hamcycle* $= \{\langle G \rangle \mid G$ is an undirected graph with a hamiltonian cycle$\}$
- *TSP* $= \{\langle G', c, k \rangle \mid G'$ is complete undirected graph, $c\colon V \times V \to \mathbb{N}$ is a cost function, $k \in \mathbb{N}$, and G' has a traveling-salesperson tour with cost at most $k\}$

$\langle G \rangle \to \langle G', c, k \rangle$:

- G' is a complete graph on $V(G)$.

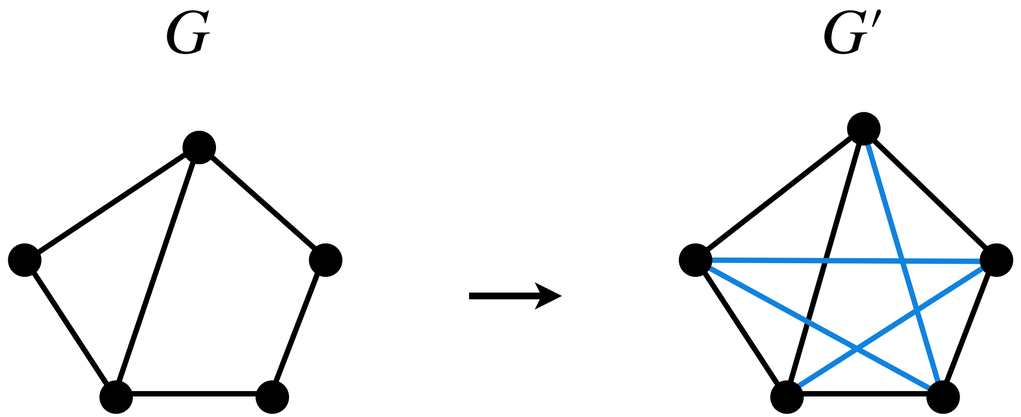


Hamcycle $\leq_p$ *TSP*

- *Hamcycle* $= \{\langle G \rangle \mid G$ is an undirected graph with a hamiltonian cycle$\}$
- *TSP* $= \{\langle G', c, k \rangle \mid G'$ is complete undirected graph, $c\colon V \times V \to \mathbb{N}$ is a cost function, $k \in \mathbb{N}$, and G' has a traveling-salesperson tour with cost at most $k\}$

$\langle G \rangle \to \langle G', c, k \rangle$:

- G' is a complete graph on $V(G)$.
- $c(\{u, v\}) =$

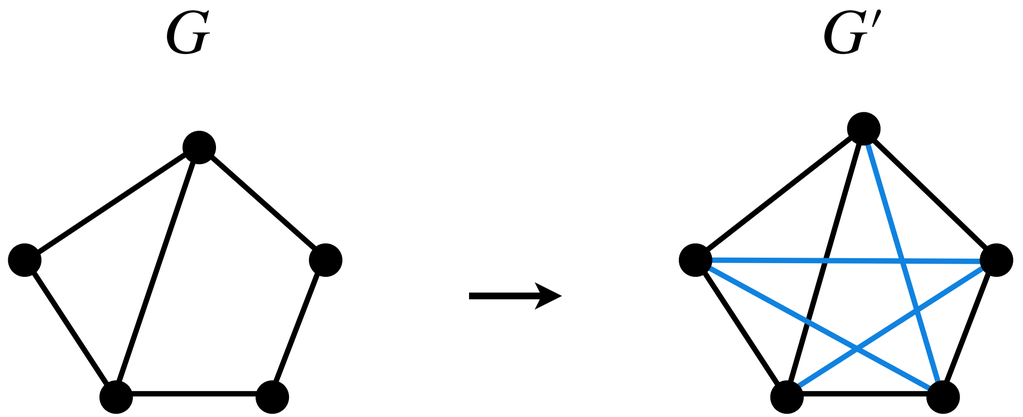


Hamcycle $\leq_p$ *TSP*

- *Hamcycle* $= \{\langle G \rangle \mid G$ is an undirected graph with a hamiltonian cycle$\}$
- *TSP* $= \{\langle G', c, k \rangle \mid G'$ is complete undirected graph, $c\colon V \times V \to \mathbb{N}$ is a cost function, $k \in \mathbb{N}$, and G' has a traveling-salesperson tour with cost at most $k\}$

$\langle G \rangle \to \langle G', c, k \rangle$:

- G' is a complete graph on $V(G)$.
- $c(\{u, v\}) = \begin{cases} \end{cases}$

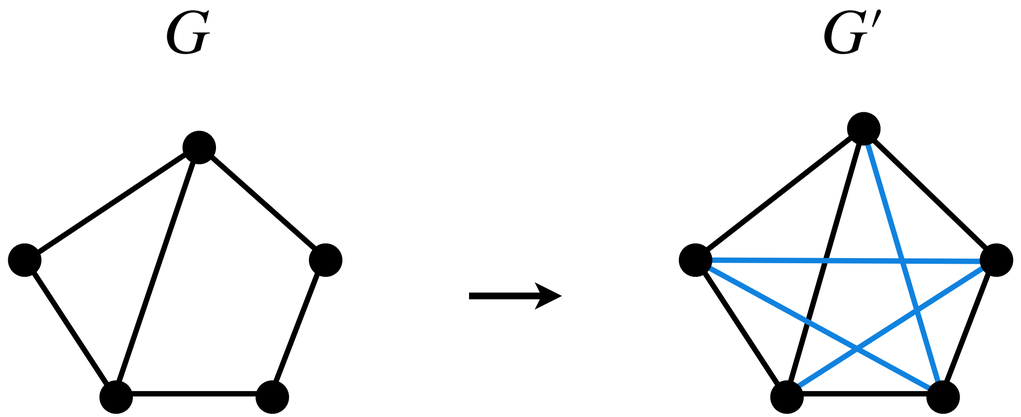


Hamcycle $\leq_p$ *TSP*

- *Hamcycle* $= \{\langle G \rangle \mid G$ is an undirected graph with a hamiltonian cycle$\}$
- *TSP* $= \{\langle G', c, k \rangle \mid G'$ is complete undirected graph, $c\colon V \times V \to \mathbb{N}$ is a cost function, $k \in \mathbb{N}$, and G' has a traveling-salesperson tour with cost at most $k\}$

$\langle G \rangle \to \langle G', c, k \rangle$:

- G' is a complete graph on $V(G)$.
- $c(\{u, v\}) = \begin{cases} 0, & \text{if } \{u, v\} \in E(G) \\ \end{cases}$

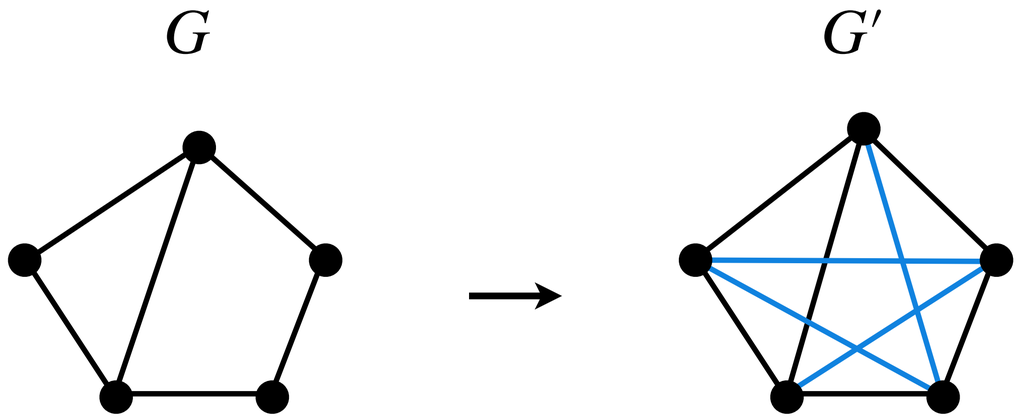


Hamcycle $\leq_p$ *TSP*

- *Hamcycle* $= \{\langle G \rangle \mid G$ is an undirected graph with a hamiltonian cycle$\}$
- *TSP* $= \{\langle G', c, k \rangle \mid G'$ is complete undirected graph, $c\colon V \times V \to \mathbb{N}$ is a cost function, $k \in \mathbb{N}$, and G' has a traveling-salesperson tour with cost at most $k\}$

$\langle G \rangle \to \langle G', c, k \rangle$:

- G' is a complete graph on $V(G)$.
- $c(\{u, v\}) = \begin{cases} 0, & \text{if } \{u, v\} \in E(G) \\ 1, & \text{if } \{u, v\} \notin E(G) \end{cases}$

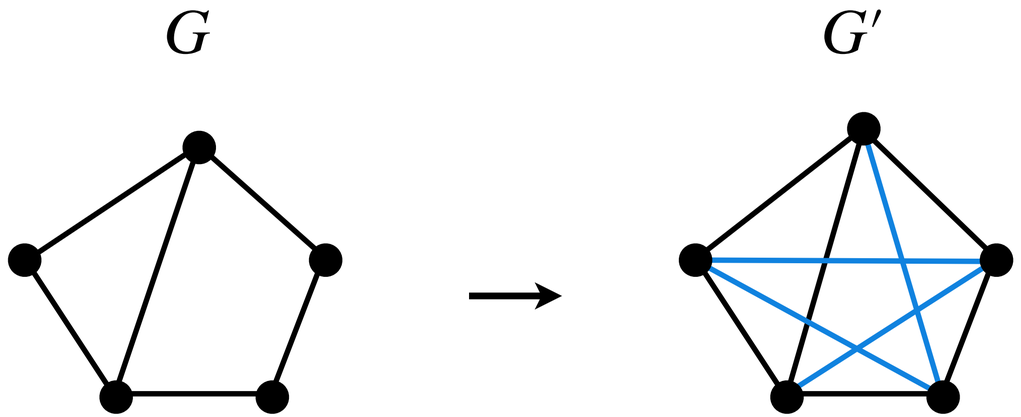


Hamcycle $\leq_p$ *TSP*

- *Hamcycle* $= \{\langle G \rangle \mid G$ is an undirected graph with a hamiltonian cycle$\}$
- *TSP* $= \{\langle G', c, k \rangle \mid G'$ is complete undirected graph, $c\colon V \times V \to \mathbb{N}$ is a cost function, $k \in \mathbb{N}$, and G' has a traveling-salesperson tour with cost at most $k\}$

$\langle G \rangle \to \langle G', c, k \rangle$:

- G' is a complete graph on $V(G)$.
- $c(\{u, v\}) = \begin{cases} 0, & \text{if } \{u, v\} \in E(G) \\ 1, & \text{if } \{u, v\} \notin E(G) \end{cases}$
- $k = 0$

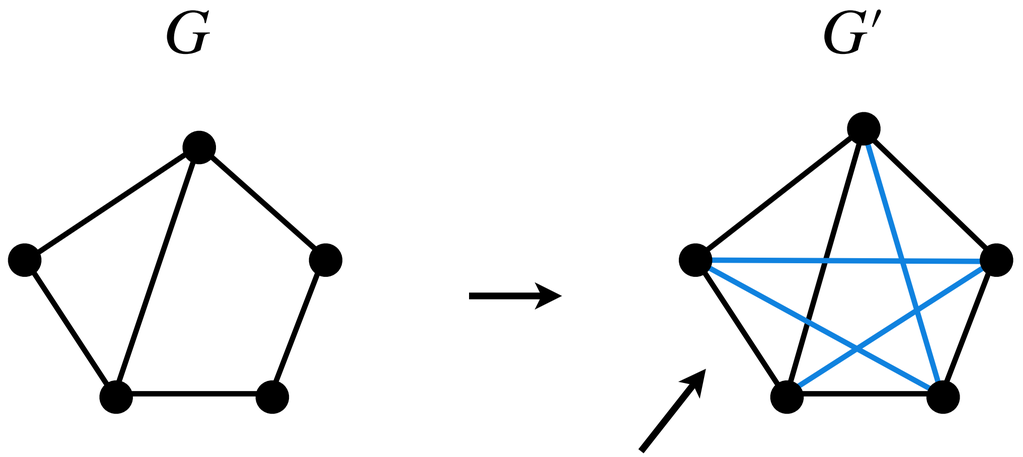


Hamcycle $\leq_p$ *TSP*

- *Hamcycle* $= \{\langle G \rangle \mid G$ is an undirected graph with a hamiltonian cycle$\}$
- *TSP* $= \{\langle G', c, k \rangle \mid G'$ is complete undirected graph, $c\colon V \times V \to \mathbb{N}$ is a cost function, $k \in \mathbb{N}$, and G' has a traveling-salesperson tour with cost at most $k\}$

$\langle G \rangle \to \langle G', c, k \rangle$:

- G' is a complete graph on $V(G)$.
- $c(\{u, v\}) = \begin{cases} 0, & \text{if } \{u, v\} \in E(G) \\ 1, & \text{if } \{u, v\} \notin E(G) \end{cases}$
- $k = 0$



Blue edges have cost 1,
black edges have cost 0

Hamcycle $\leq_p$ *TSP*

$\langle G \rangle \to \langle G', c, k \rangle$:

- G' is a complete graph on $V(G)$.
- $c(\{u, v\}) = \begin{cases} 0, & \text{if } \{u, v\} \in E(G) \\ 1, & \text{if } \{u, v\} \notin E(G) \end{cases}$
- $k = 0$

Hamcycle $\leq_p$ *TSP*

$\langle G \rangle \to \langle G', c, k \rangle$:

- G' is a complete graph on $V(G)$.
- $c(\{u, v\}) = \begin{cases} 0, & \text{if } \{u, v\} \in E(G) \\ 1, & \text{if } \{u, v\} \notin E(G) \end{cases}$
- $k = 0$

Claim: G contains a Hamiltonian cycle if and only if G' has TSP tour with cost ≤ 0.

Hamcycle $\leq_p$ *TSP*

$\langle G \rangle \to \langle G', c, k \rangle$:

- G' is a complete graph on $V(G)$.
- $c(\{u, v\}) = \begin{cases} 0, & \text{if } \{u, v\} \in E(G) \\ 1, & \text{if } \{u, v\} \notin E(G) \end{cases}$
- $k = 0$

Claim: G contains a Hamiltonian cycle if and only if G' has TSP tour with cost ≤ 0.

Proof: ($\Longrightarrow$)

Hamcycle $\leq_p$ *TSP*

$\langle G \rangle \to \langle G', c, k \rangle$:

- G' is a complete graph on $V(G)$.
- $c(\{u, v\}) = \begin{cases} 0, & \text{if } \{u, v\} \in E(G) \\ 1, & \text{if } \{u, v\} \notin E(G) \end{cases}$
- $k = 0$

Claim: G contains a Hamiltonian cycle if and only if G' has TSP tour with cost ≤ 0.

Proof: ($\Longrightarrow$) If G contains a hamiltonian cycle C

Hamcycle $\leq_p$ *TSP*

$\langle G \rangle \to \langle G', c, k \rangle$:

- G' is a complete graph on $V(G)$.
- $c(\{u, v\}) = \begin{cases} 0, & \text{if } \{u, v\} \in E(G) \\ 1, & \text{if } \{u, v\} \notin E(G) \end{cases}$
- $k = 0$

Claim: G contains a Hamiltonian cycle if and only if G' has TSP tour with cost ≤ 0.

Proof: ($\Longrightarrow$) If G contains a hamiltonian cycle C, then C in G' will be a TSP tour with cost 0.

Hamcycle $\leq_p$ *TSP*

$\langle G \rangle \to \langle G', c, k \rangle$:

- G' is a complete graph on $V(G)$.
- $c(\{u, v\}) = \begin{cases} 0, & \text{if } \{u, v\} \in E(G) \\ 1, & \text{if } \{u, v\} \notin E(G) \end{cases}$
- $k = 0$

Claim: G contains a Hamiltonian cycle if and only if G' has TSP tour with cost ≤ 0.

Proof: ($\Longleftarrow$)

Hamcycle $\leq_p$ *TSP*

$\langle G \rangle \to \langle G', c, k \rangle$:

- G' is a complete graph on $V(G)$.
- $c(\{u, v\}) = \begin{cases} 0, & \text{if } \{u, v\} \in E(G) \\ 1, & \text{if } \{u, v\} \notin E(G) \end{cases}$
- $k = 0$

Claim: G contains a Hamiltonian cycle if and only if G' has TSP tour with cost ≤ 0.

Proof: ($\Longleftarrow$) Suppose G' contains a TSP tour C of cost 0.

Hamcycle $\leq_p$ *TSP*

$\langle G \rangle \to \langle G', c, k \rangle$:

- G' is a complete graph on $V(G)$.
- $c(\{u, v\}) = \begin{cases} 0, & \text{if } \{u, v\} \in E(G) \\ 1, & \text{if } \{u, v\} \notin E(G) \end{cases}$
- $k = 0$

Claim: G contains a Hamiltonian cycle if and only if G' has TSP tour with cost ≤ 0.

Proof: ($\Longleftarrow$) Suppose G' contains a TSP tour C of cost 0.

Then, every edge of C will have cost 0.

Hamcycle $\leq_p$ *TSP*

$\langle G \rangle \to \langle G', c, k \rangle$:

- G' is a complete graph on $V(G)$.
- $c(\{u, v\}) = \begin{cases} 0, & \text{if } \{u, v\} \in E(G) \\ 1, & \text{if } \{u, v\} \notin E(G) \end{cases}$
- $k = 0$

Claim: G contains a Hamiltonian cycle if and only if G' has TSP tour with cost ≤ 0.

Proof: ($\Longleftarrow$) Suppose G' contains a TSP tour C of cost 0.

Then, every edge of C will have cost 0.

Hence, every edge of C must be in G as well.

Hamcycle $\leq_p$ *TSP*

$\langle G \rangle \to \langle G', c, k \rangle$:

- G' is a complete graph on $V(G)$.
- $c(\{u, v\}) = \begin{cases} 0, & \text{if } \{u, v\} \in E(G) \\ 1, & \text{if } \{u, v\} \notin E(G) \end{cases}$
- $k = 0$

Claim: G contains a Hamiltonian cycle if and only if G' has TSP tour with cost ≤ 0.

Proof: ($\Longleftarrow$) Suppose G' contains a TSP tour C of cost 0.

Then, every edge of C will have cost 0.

Hence, every edge of C must be in G as well.

Therefore, C is also a hamiltonian cycle in G.

Hamcycle $\leq_p$ *TSP*

$\langle G \rangle \to \langle G', c, k \rangle$:

- G' is a complete graph on $V(G)$.
- $c(\{u, v\}) = \begin{cases} 0, & \text{if } \{u, v\} \in E(G) \\ 1, & \text{if } \{u, v\} \notin E(G) \end{cases}$
- $k = 0$

Claim: G contains a Hamiltonian cycle if and only if G' has TSP tour with cost ≤ 0.

Proof: ($\Longleftarrow$) Suppose G' contains a TSP tour C of cost 0.

Then, every edge of C will have cost 0.

Hence, every edge of C must be in G as well.

Therefore, C is also a hamiltonian cycle in G.

■

OptVertexCover

OptVertexCover

OptVertexCover:

OptVertexCover

OptVertexCover:

Input: A graph an undirected graph G.

OptVertexCover

OptVertexCover:

Input: A graph an undirected graph G.

Output: Find a vertex cover of minimum size in G.

OptVertexCover

OptVertexCover:

Input: A graph an undirected graph G.

Output: Find a vertex cover of minimum size in G.

VertexCover:

OptVertexCover

OptVertexCover:

Input: A graph an undirected graph G.

Output: Find a vertex cover of minimum size in G.

VertexCover:

Input: A graph an undirected graph G and integer $k \geq 0$.

OptVertexCover

OptVertexCover:

Input: A graph an undirected graph G.

Output: Find a vertex cover of minimum size in G.

VertexCover:

Input: A graph an undirected graph G and integer $k \geq 0$.

Output: Find whether G has a vertex cover of size k.

OptVertexCover

OptVertexCover:

Input: A graph an undirected graph G.

Output: Find a vertex cover of minimum size in G.

We know *VertexCover* is NP-complete.

VertexCover:

Input: A graph an undirected graph G and integer $k \geq 0$.

Output: Find whether G has a vertex cover of size k.

OptVertexCover

OptVertexCover

Claim: If *OptVertexCover* is polytime solvable, then *VertexCover* is polytime solvable.

OptVertexCover

Claim: If *OptVertexCover* is polytime solvable, then *VertexCover* is polytime solvable.

Proof: Suppose *OptVertexCover* has a polytime algorithm A.

OptVertexCover

Claim: If *OptVertexCover* is polytime solvable, then *VertexCover* is polytime solvable.

Proof: Suppose *OptVertexCover* has a polytime algorithm A.

Then polytime algorithm B for *VertexCover* on input (G, k):

OptVertexCover

Claim: If *OptVertexCover* is polytime solvable, then *VertexCover* is polytime solvable.

Proof: Suppose *OptVertexCover* has a polytime algorithm A.

Then polytime algorithm B for *VertexCover* on input (G, k):

1. Finds minimum size vertex cover, $S = A(G)$.

OptVertexCover

Claim: If *OptVertexCover* is polytime solvable, then *VertexCover* is polytime solvable.

Proof: Suppose *OptVertexCover* has a polytime algorithm A.

Then polytime algorithm B for *VertexCover* on input (G, k):

1. Finds minimum size vertex cover, $S = A(G)$.
2. If $k \geq |S|$, output "Yes".

OptVertexCover

Claim: If *OptVertexCover* is polytime solvable, then *VertexCover* is polytime solvable.

Proof: Suppose *OptVertexCover* has a polytime algorithm A.

Then polytime algorithm B for *VertexCover* on input (G, k):

1. Finds minimum size vertex cover, $S = A(G)$.
2. If $k \geq |S|$, output "Yes".
3. If $k < |S|$, output "No".

OptVertexCover

Claim: If *OptVertexCover* is polytime solvable, then *VertexCover* is polytime solvable.

Proof: Suppose *OptVertexCover* has a polytime algorithm A.

Then polytime algorithm B for *VertexCover* on input (G, k):

1. Finds minimum size vertex cover, $S = A(G)$.
2. If $k \geq |S|$, output "Yes".
3. If $k < |S|$, output "No".

We can add $k - |S|$ more vertices to S to get a vertex cover of size k.

OptVertexCover

Claim: If *OptVertexCover* is polytime solvable, then *VertexCover* is polytime solvable.

Proof: Suppose *OptVertexCover* has a polytime algorithm A.

Then polytime algorithm B for *VertexCover* on input (G, k):

1. Finds minimum size vertex cover, $S = A(G)$.
2. If $k \geq |S|$, output "Yes".
3. If $k < |S|$, output "No".

OptVertexCover

Claim: If *OptVertexCover* is polytime solvable, then *VertexCover* is polytime solvable.

Proof: Suppose *OptVertexCover* has a polytime algorithm A.

Then polytime algorithm B for *VertexCover* on input (G, k):

1. Finds minimum size vertex cover, $S = A(G)$.
2. If $k \geq |S|$, output "Yes".
3. If $k < |S|$, output "No".

There cannot be a vertex cover smaller than the minimum vertex cover.

OptVertexCover

Claim: If *OptVertexCover* is polytime solvable, then *VertexCover* is polytime solvable.

Proof: Suppose *OptVertexCover* has a polytime algorithm A.

Then polytime algorithm B for *VertexCover* on input (G, k):

1. Finds minimum size vertex cover, $S = A(G)$.
2. If $k \geq |S|$, output "Yes".
3. If $k < |S|$, output "No".

There cannot be a vertex cover smaller than the minimum vertex cover.

■

OptVertexCover

OptVertexCover

Claim: If *VertexCover* is polytime solvable, then *OptVertexCover* is polytime solvable.

OptVertexCover

Claim: If *VertexCover* is polytime solvable, then *OptVertexCover* is polytime solvable.

Proof: Suppose *VertexCover* has a polytime algorithm A.

OptVertexCover

Claim: If *VertexCover* is polytime solvable, then *OptVertexCover* is polytime solvable.

Proof: Suppose *VertexCover* has a polytime algorithm A.

Then polytime algorithm B for *OptVertexCover* on input G:

OptVertexCover

Claim: If *VertexCover* is polytime solvable, then *OptVertexCover* is polytime solvable.

Proof: Suppose *VertexCover* has a polytime algorithm A.

Then polytime algorithm B for *OptVertexCover* on input G:

1. Starts from $k = 0$.

OptVertexCover

Claim: If *VertexCover* is polytime solvable, then *OptVertexCover* is polytime solvable.

Proof: Suppose *VertexCover* has a polytime algorithm A.

Then polytime algorithm B for *OptVertexCover* on input G:

1. Starts from $k = 0$.
2. Checks whether G has vertex cover of size k using $A(G, k)$.

OptVertexCover

Claim: If *VertexCover* is polytime solvable, then *OptVertexCover* is polytime solvable.

Proof: Suppose *VertexCover* has a polytime algorithm A.

Then polytime algorithm B for *OptVertexCover* on input G:

1. Starts from $k = 0$.
2. Checks whether G has vertex cover of size k using $A(G, k)$.
3. If $A(G, k)$ returns **"Yes"**, then output k.

OptVertexCover

Claim: If *VertexCover* is polytime solvable, then *OptVertexCover* is polytime solvable.

Proof: Suppose *VertexCover* has a polytime algorithm A.

Then polytime algorithm B for *OptVertexCover* on input G:

1. Starts from $k = 0$.
2. Checks whether G has vertex cover of size k using $A(G, k)$.
3. If $A(G, k)$ returns **"Yes"**, then output k.
4. If $A(G, k)$ returns **"No"**, increase k by 1 and go to Step 2.

OptVertexCover

Claim: If ***VertexCover*** is polytime solvable, then ***OptVertexCover*** is polytime solvable.

Proof: Suppose ***VertexCover*** has a polytime algorithm A.

Then polytime algorithm B for ***OptVertexCover*** on input G:

1. Starts from $k = 0$.
2. Checks whether G has vertex cover of size k using $A(G, k)$.
3. If $A(G, k)$ returns **"Yes"**, then output k.
4. If $A(G, k)$ returns **"No"**, increase k by 1 and go to Step 2.

■

OptVertexCover

OptVertexCover:

Input: A graph an undirected graph G.

Output: Find a vertex cover of minimum size in G.

VertexCover:

Input: A graph an undirected graph G and integer $k \geq 0$.

Output: Find whether G has a vertex cover of size k.

OptVertexCover

While polytime exact algorithm is unlikely,
we may still have a decent polynomial approximation algorithm.

OptVertexCover:

Input: A graph an undirected graph G.

Output: Find a vertex cover of minimum size in G.

VertexCover:

Input: A graph an undirected graph G and integer $k \geq 0$.

Output: Find whether G has a vertex cover of size k.

Approximation Algorithms

Approximation Algorithms

Defn: Let P be an optimization problem.

Approximation Algorithms

Defn: Let P be an optimization problem. For every input x of P, let $OPT(x)$ denote the value

Approximation Algorithms

Defn: Let P be an optimization problem. For every input x of P, let $OPT(x)$ denote the value of the **optimal solution** for x.

Approximation Algorithms

Defn: Let P be an optimization problem. For every input x of P, let $OPT(x)$ denote the value of the **optimal solution** for x. Let A be an algorithm and ρ a constant such that $\rho \geq 1$.

Approximation Algorithms

Defn: Let P be an optimization problem. For every input x of P, let $OPT(x)$ denote the value of the **optimal solution** for x. Let A be an algorithm and ρ a constant such that $\rho \geq 1$.

- If P is a **minimisation** problem, then A is ρ-**approximation** for P if for every input x:

Approximation Algorithms

Defn: Let P be an optimization problem. For every input x of P, let $OPT(x)$ denote the value of the **optimal solution** for x. Let A be an algorithm and ρ a constant such that $\rho \geq 1$.

- If P is a **minimisation** problem, then A is **ρ-approximation** for P if for every input x:

$$OPT(x) \leq A(x) \leq \rho \, . \, OPT(x)$$

Approximation Algorithms

Defn: Let P be an optimization problem. For every input x of P, let $OPT(x)$ denote the value of the **optimal solution** for x. Let A be an algorithm and ρ a constant such that $\rho \geq 1$.

- If P is a **minimisation** problem, then A is ρ-**approximation** for P if for every input x:

$$OPT(x) \leq A(x) \leq \rho \, . \, OPT(x)$$

A 2-approximation algorithm for minimum vertex cover problem,

Approximation Algorithms

Defn: Let P be an optimization problem. For every input x of P, let $OPT(x)$ denote the value of the **optimal solution** for x. Let A be an algorithm and ρ a constant such that $\rho \geq 1$.

- If P is a **minimisation** problem, then A is **ρ-approximation** for P if for every input x:

$$OPT(x) \leq A(x) \leq \rho \, . \, OPT(x)$$

A 2-approximation algorithm for minimum vertex cover problem,
will give a vertex cover of size k to $2k$, if minimum vertex cover's size is k.

Approximation Algorithms

Defn: Let P be an optimization problem. For every input x of P, let $OPT(x)$ denote the value of the **optimal solution** for x. Let A be an algorithm and ρ a constant such that $\rho \geq 1$.

- If P is a **minimisation** problem, then A is **ρ-approximation** for P if for every input x:

$$OPT(x) \leq A(x) \leq \rho \, . \, OPT(x)$$

Approximation Algorithms

Defn: Let P be an optimization problem. For every input x of P, let $OPT(x)$ denote the value of the **optimal solution** for x. Let A be an algorithm and ρ a constant such that $\rho \geq 1$.

- If P is a **minimisation** problem, then A is ρ-**approximation** for P if for every input x:

$$OPT(x) \leq A(x) \leq \rho \, . \, OPT(x)$$

- If P is a **maximisation** problem, then A is ρ-**approximation** for P if for every input x:

Approximation Algorithms

Defn: Let P be an optimization problem. For every input x of P, let $OPT(x)$ denote the value of the **optimal solution** for x. Let A be an algorithm and ρ a constant such that $\rho \geq 1$.

- If P is a **minimisation** problem, then A is ρ-**approximation** for P if for every input x:

$$OPT(x) \leq A(x) \leq \rho \, . \, OPT(x)$$

- If P is a **maximisation** problem, then A is ρ-**approximation** for P if for every input x:

$$\frac{OPT(x)}{\rho} \leq A(x) \leq OPT(x)$$

Approximation Algorithms

Defn: Let P be an optimization problem. For every input x of P, let $OPT(x)$ denote the value of the **optimal solution** for x. Let A be an algorithm and ρ a constant such that $\rho \geq 1$.

- If P is a **minimisation** problem, then A is ρ-**approximation** for P if for every input x:

$$OPT(x) \leq A(x) \leq \rho \, . \, OPT(x)$$

- If P is a **maximisation** problem, then A is ρ-**approximation** for P if for every input x:

$$\frac{OPT(x)}{\rho} \leq A(x) \leq OPT(x)$$

A 2-approximation algorithm for maximum independent set problem,

Approximation Algorithms

Defn: Let P be an optimization problem. For every input x of P, let $OPT(x)$ denote the value of the **optimal solution** for x. Let A be an algorithm and ρ a constant such that $\rho \geq 1$.

- If P is a **minimisation** problem, then A is ρ-**approximation** for P if for every input x:

$$OPT(x) \leq A(x) \leq \rho \, . \, OPT(x)$$

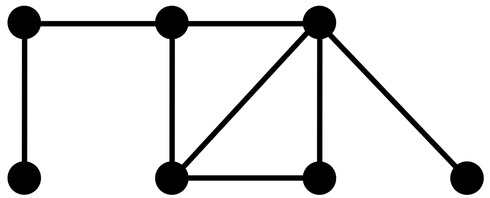
- If P is a **maximisation** problem, then A is ρ-**approximation** for P if for every input x:

$$\frac{OPT(x)}{\rho} \leq A(x) \leq OPT(x)$$

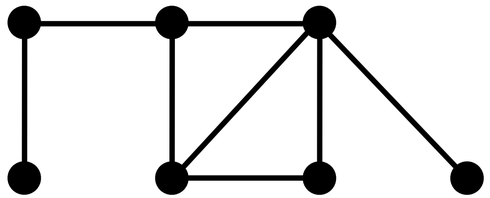
A 2-approximation algorithm for maximum independent set problem,
will give an independent set of size $k/2$ to k, if maximum independent set's size is k.

2-approximation for *OptVertexCover*

2-approximation for *OptVertexCover*

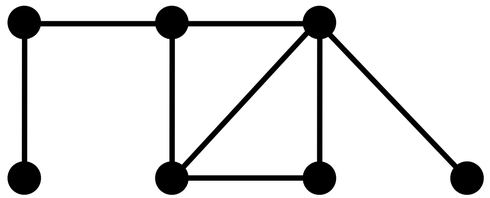


2-approximation for *OptVertexCover*



Idea:

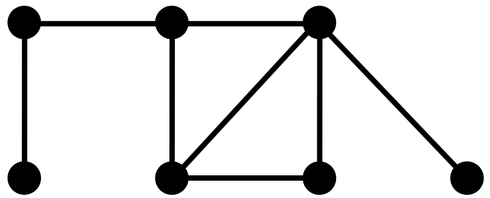
2-approximation for *OptVertexCover*



Idea:

- Pick any edge $\{u, v\}$.

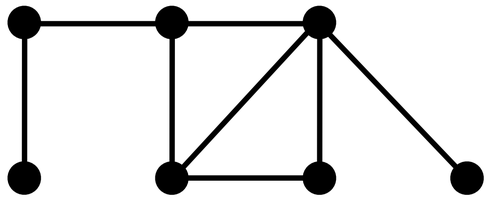
2-approximation for *OptVertexCover*



Idea:

- Pick any edge $\{u, v\}$.
- Add u and v to the vertex cover.

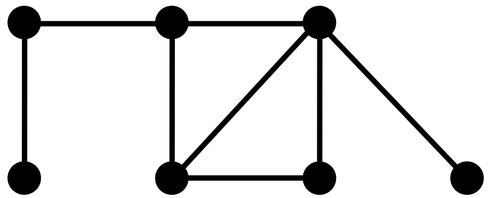
2-approximation for *OptVertexCover*



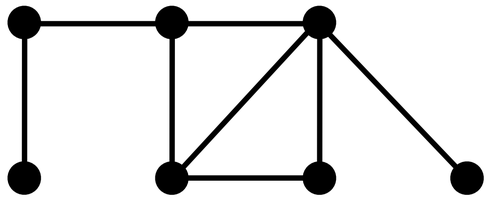
Idea:

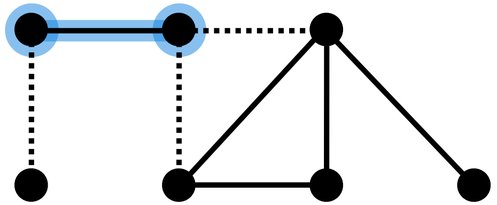
- Pick any edge $\{u, v\}$.
- Add u and v to the vertex cover.
- Delete the edges incident on u or v.

2-approximation for *OptVertexCover*

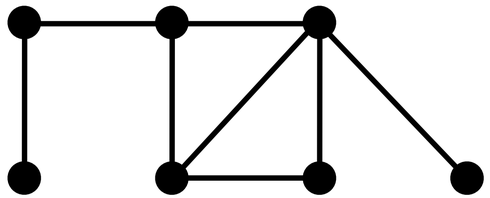


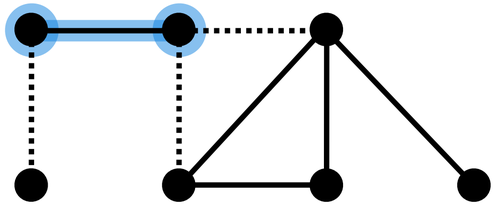
2-approximation for *OptVertexCover*

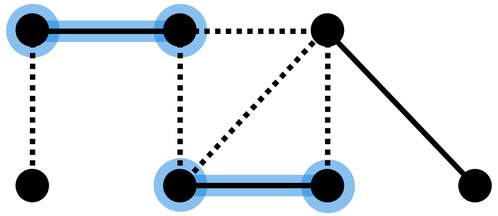




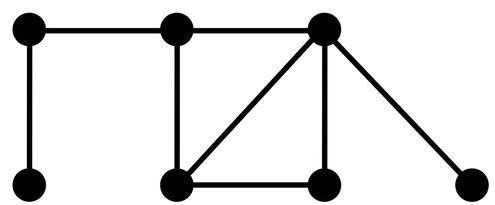
2-approximation for *OptVertexCover*

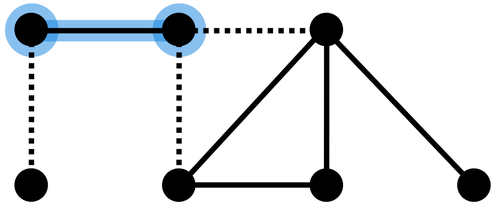


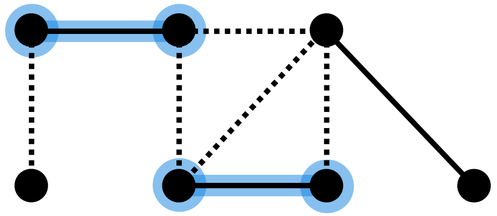


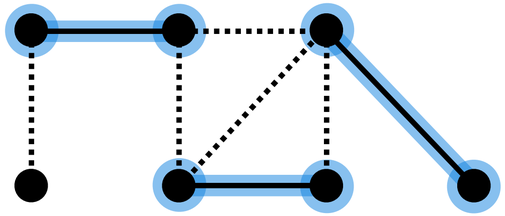


2-approximation for *OptVertexCover*

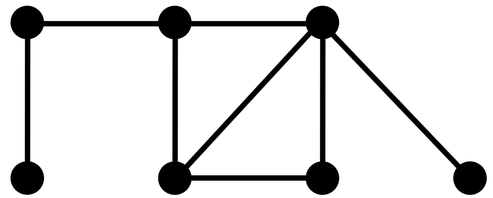


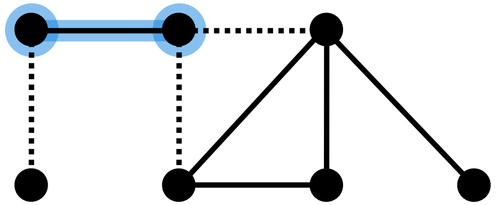


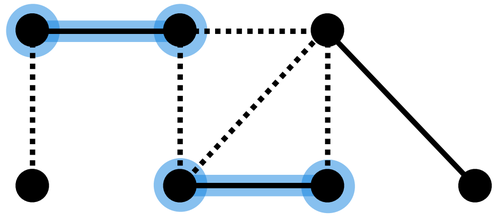


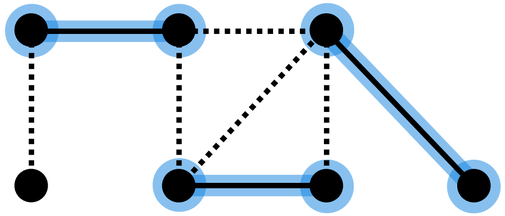


2-approximation for *OptVertexCover*

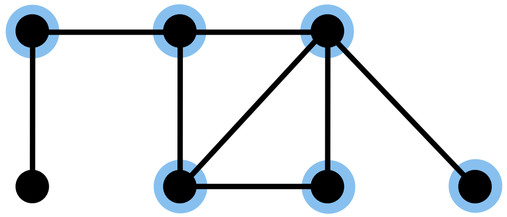




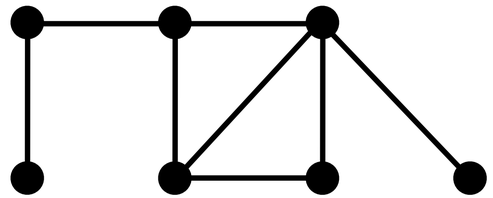


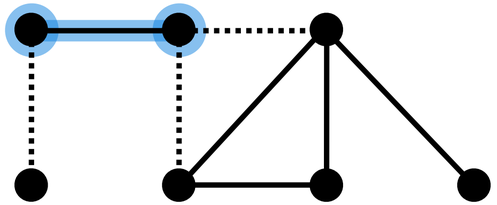


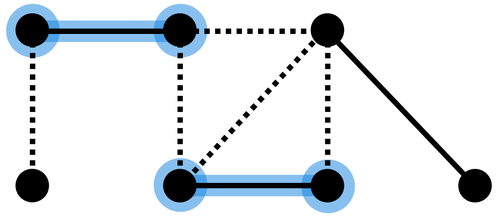
Vertex cover produced by the algorithm.

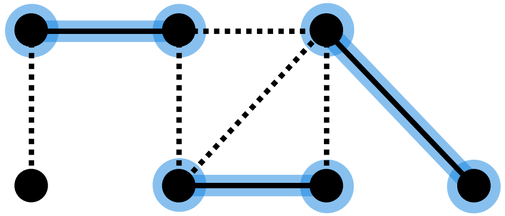


2-approximation for *OptVertexCover*

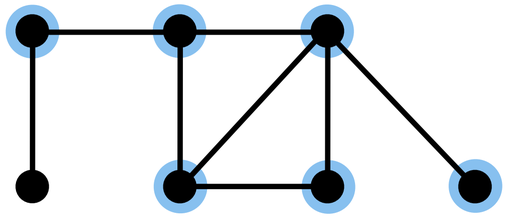


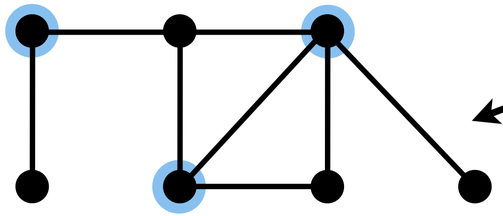






Vertex cover produced by the algorithm.





Minimum size vertex cover

2-approximation for *OptVertexCover*

2-approximation for *OptVertexCover*

Approx-VC(G)

2-approximation for *OptVertexCover*

Approx-VC(G)

1. $C = \emptyset$ *// C will be the computed vertex cover*

2-approximation for *OptVertexCover*

Approx-VC(G)

1. $C = \emptyset$ *// C will be the computed vertex cover*
2. $E' = E(G)$

2-approximation for *OptVertexCover*

Approx-VC(G)

1. $C = \emptyset$ *// C will be the computed vertex cover*
2. $E' = E(G)$
3. **while** $E' \neq \emptyset$

2-approximation for *OptVertexCover*

Approx-VC(G)

1. $C = \emptyset$ *// C will be the computed vertex cover*
2. $E' = E(G)$
3. **while** $E' \neq \emptyset$
4. let $\{u, v\}$ be an arbitrary edge of E'

2-approximation for *OptVertexCover*

Approx-VC(G)

1. $C = \emptyset$ *// C will be the computed vertex cover*
2. $E' = E(G)$
3. **while** $E' \neq \emptyset$
4. let $\{u, v\}$ be an arbitrary edge of E'
5. $C = C \cup \{u, v\}$ *// add u and v to vertex cover*

2-approximation for *OptVertexCover*

Approx-VC(G)

1. $C = \emptyset$ *// C will be the computed vertex cover*
2. $E' = E(G)$
3. **while** $E' \neq \emptyset$
4. let $\{u, v\}$ be an arbitrary edge of E'
5. $C = C \cup \{u, v\}$ *// add u and v to vertex cover*
6. remove those edges from E' which are incident on u or v

2-approximation for *OptVertexCover*

Approx-VC(G)

1. $C = \emptyset$ *// C will be the computed vertex cover*
2. $E' = E(G)$
3. **while** $E' \neq \emptyset$
4. let $\{u, v\}$ be an arbitrary edge of E'
5. $C = C \cup \{u, v\}$ *// add u and v to vertex cover*
6. remove those edges from E' which are incident on u or v
7. **return** C

2-approximation for *OptVertexCover*

Approx-VC(G)

1. $C = \emptyset$ *// C will be the computed vertex cover*
2. $E' = E(G)$
3. **while** $E' \neq \emptyset$
4. let $\{u, v\}$ be an arbitrary edge of E'
5. $C = C \cup \{u, v\}$ *// add u and v to vertex cover*
6. remove those edges from E' which are incident on u or v
7. **return** C

Time Complexity: Polynomial in the size of G.

2-approximation for *OptVertexCover*

2-approximation for *OptVertexCover*

Claim: **Approx-VC**(G) is a 2-approximation algorithm for OptVertexCover.

2-approximation for *OptVertexCover*

Claim: **Approx-VC**(G) is a 2-approximation algorithm for OptVertexCover.

Proof:

2-approximation for *OptVertexCover*

Claim: $\textbf{Approx-VC}(G)$ is a 2-approximation algorithm for **OptVertexCover**.

Proof: It's easy to see that C will be a vertex cover.

2-approximation for *OptVertexCover*

Claim: $\textbf{Approx-VC}(G)$ is a 2-approximation algorithm for **OptVertexCover**.

Proof: It's easy to see that C will be a vertex cover.

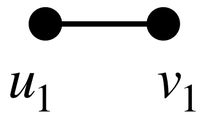
Suppose vertices from the following sequence of edges were added to C:

2-approximation for *OptVertexCover*

Claim: **Approx-VC**(G) is a 2-approximation algorithm for **OptVertexCover**.

Proof: It's easy to see that C will be a vertex cover.

Suppose vertices from the following sequence of edges were added to C:



2-approximation for *OptVertexCover*

Claim: **Approx-VC**(G) is a 2-approximation algorithm for **OptVertexCover**.

Proof: It's easy to see that C will be a vertex cover.

Suppose vertices from the following sequence of edges were added to C:

2-approximation for *OptVertexCover*

Claim: **Approx-VC**(G) is a 2-approximation algorithm for **OptVertexCover**.

Proof: It's easy to see that C will be a vertex cover.

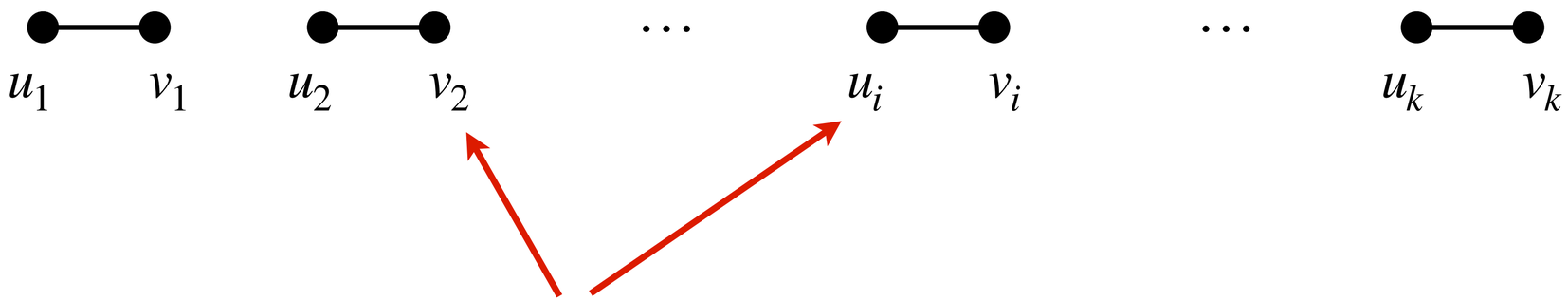
Suppose vertices from the following sequence of edges were added to C:

2-approximation for *OptVertexCover*

Claim: **Approx-VC**(G) is a 2-approximation algorithm for **OptVertexCover**.

Proof: It's easy to see that C will be a vertex cover.

Suppose vertices from the following sequence of edges were added to C:

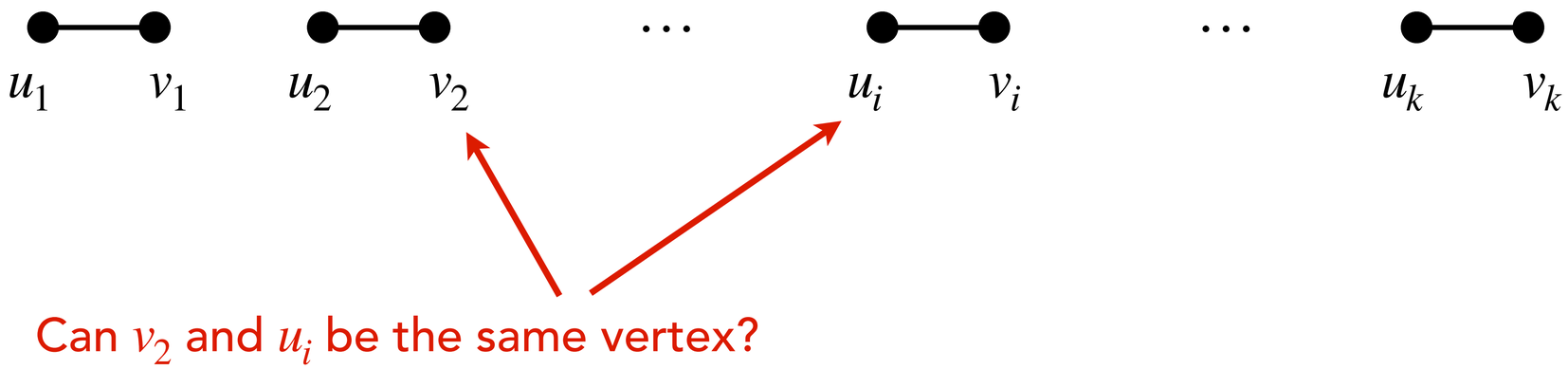


2-approximation for *OptVertexCover*

Claim: **Approx-VC**(G) is a 2-approximation algorithm for **OptVertexCover**.

Proof: It's easy to see that C will be a vertex cover.

Suppose vertices from the following sequence of edges were added to C:

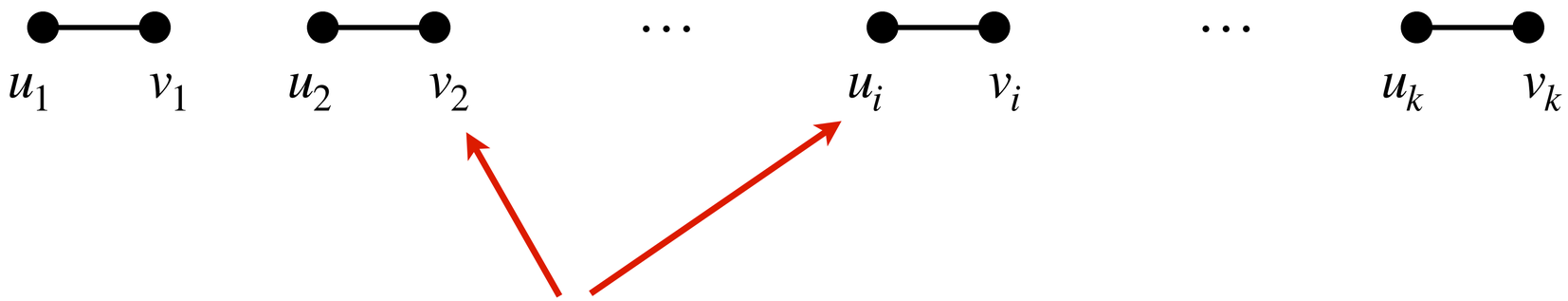


2-approximation for *OptVertexCover*

Claim: **Approx-VC**(G) is a 2-approximation algorithm for **OptVertexCover**.

Proof: It's easy to see that C will be a vertex cover.

Suppose vertices from the following sequence of edges were added to C:



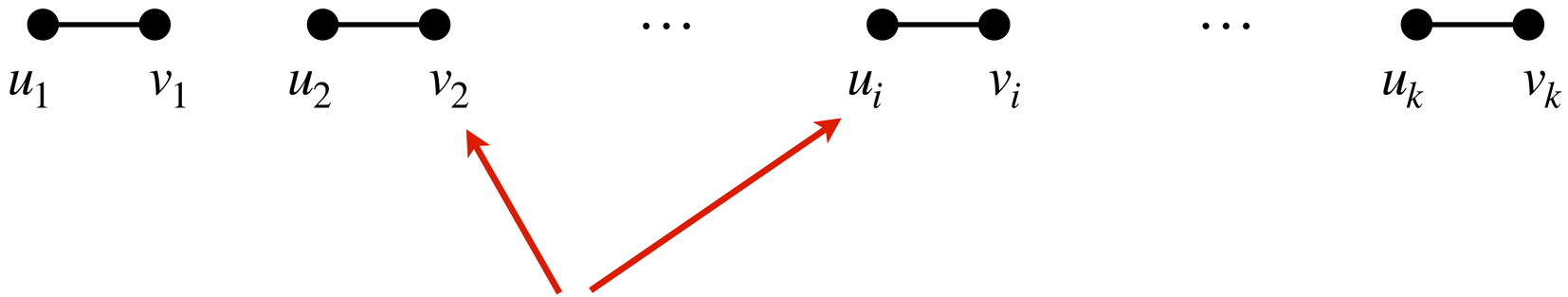
Can v_2 and u_i be the same vertex? No, because when we added

2-approximation for *OptVertexCover*

Claim: **Approx-VC**(G) is a 2-approximation algorithm for **OptVertexCover**.

Proof: It's easy to see that C will be a vertex cover.

Suppose vertices from the following sequence of edges were added to C:



Can v_2 and u_i be the same vertex? No, because when we added $\{u_2, v_2\}$ to C, we deleted all the edges incident on u_2 and v_2 from E'.

2-approximation for *OptVertexCover*

Claim: **Approx-VC**(G) is a 2-approximation algorithm for **OptVertexCover**.

Proof: It's easy to see that C will be a vertex cover.

Suppose vertices from the following sequence of edges were added to C:

2-approximation for *OptVertexCover*

Claim: **Approx-VC**(G) is a 2-approximation algorithm for **OptVertexCover**.

Proof: It's easy to see that C will be a vertex cover.

Suppose vertices from the following sequence of edges were added to C:

Then, no two edges in the above sequence can have a common vertex.

2-approximation for *OptVertexCover*

Claim: **Approx-VC**(G) is a 2-approximation algorithm for **OptVertexCover**.

Proof: It's easy to see that C will be a vertex cover.

Suppose vertices from the following sequence of edges were added to C:

Then, no two edges in the above sequence can have a common vertex.

Clearly, $|C| = 2k$.

2-approximation for *OptVertexCover*

Claim: **Approx-VC**(G) is a 2-approximation algorithm for **OptVertexCover**.

Proof: It's easy to see that C will be a vertex cover.

Suppose vertices from the following sequence of edges were added to C:

Then, no two edges in the above sequence can have a common vertex.

Clearly, $|C| = 2k$. Let C' be the minimum size vertex cover.

2-approximation for *OptVertexCover*

Claim: **Approx-VC**(G) is a 2-approximation algorithm for **OptVertexCover**.

Proof: It's easy to see that C will be a vertex cover.

Suppose vertices from the following sequence of edges were added to C:

Then, no two edges in the above sequence can have a common vertex.

Clearly, $|C| = 2k$. Let C' be the minimum size vertex cover.

C' must have at least one vertex in each edge of the above sequence.

2-approximation for *OptVertexCover*

Claim: **Approx-VC**(G) is a 2-approximation algorithm for **OptVertexCover**.

Proof: It's easy to see that C will be a vertex cover.

Suppose vertices from the following sequence of edges were added to C:

Then, no two edges in the above sequence can have a common vertex.

Clearly, $|C| = 2k$. Let C' be the minimum size vertex cover.

C' must have at least one vertex in each edge of the above sequence.

Therefore, $|C'| \geq k$.

2-approximation for *OptVertexCover*

Claim: **Approx-VC**(G) is a 2-approximation algorithm for **OptVertexCover**.

Proof: It's easy to see that C will be a vertex cover.

Suppose vertices from the following sequence of edges were added to C:

Then, no two edges in the above sequence can have a common vertex.

Clearly, $|C| = 2k$. Let C' be the minimum size vertex cover.

C' must have at least one vertex in each edge of the above sequence.

Therefore, $|C'| \geq k$.

■